SKALA

+ POSTGRESQL 기반 DDL/DML 설계 및 조회 실습

스마트 데이터의 이해

9반 P282 김민정

스마트 데이터의 이해

학사관리시스템에 대한 요구사항 (설계 방향)

본 시스템은 대학의 학사 정보를 통합 관리하기 위한 데이터베이스로, 교수·강좌·학생·수강신청 4개의 핵심 엔터티를 중심으로 설계하였다. 설계 시 반영한 요구사항은 다음과 같다.

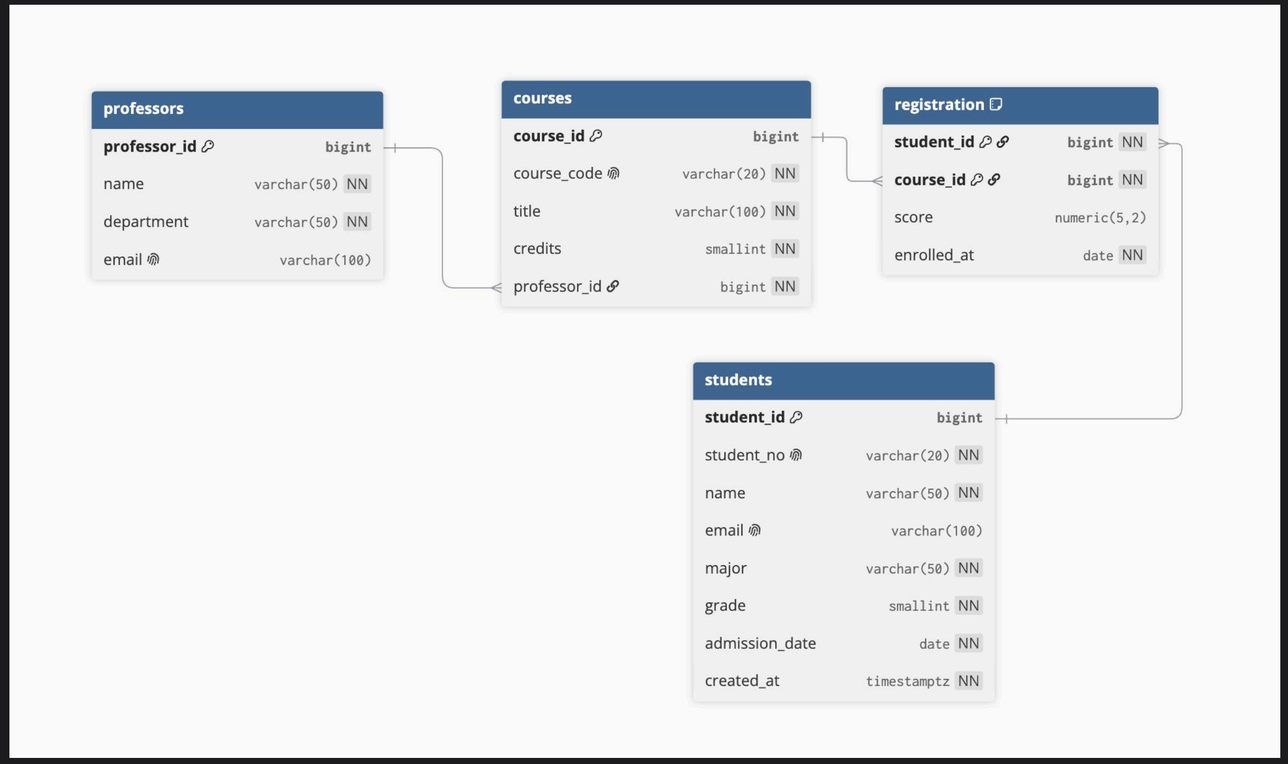
엔티티별 요구사항

- 교수(professors): 이름과 소속 학과는 필수이며, 이메일은 등록되어 있는 경우 시스템 전체에서 유일해야 한다.
- 강좌(courses): 강좌코드는 유일해야 하며, 학점은 1~3학점 범위로 제한한다. 모든 강좌는 반드시 담당 교수를 가져야 하며, 담당 강좌가 남아 있는 교수는 삭제할 수 없다(ON DELETE RESTRICT).
- 학생(students): 학번은 유일해야 하지만, 이메일은 아직 등록하지 않은 학생을 고려하여 NULL을 허용한다. 학년은 1~4 범위로 제한하고, 입학일과 최초 등록 시각을 함께 관리한다
- 수강신청(registration): 동일 학생이 동일 강좌를 중복 신청할 수 없도록 (student_id, course_id) 를 복합 기본키로 지정한다. 성적은 미채점 상태(NULL)를 허용하되, 입력되는 경우 0~100점 범위를 벗어날 수 없다. 학생 또는 강좌가 삭제되면 관련 수강신청 기록도 함께 삭제한다(ON DELETE CASCADE).

스마트 데이터의 이해

ERD

professors, courses, students, registration ERD



diagram

요구사항에 따라 4개 테이블과 기본키/외래키/제약조건을 다음과 같이 설계하였다

professors와 courses 는 1:N

students와 registration은 1:N

courses와 registration은 1:N 관계이며, registration은 students와 courses 사이의 다대다(M:N) 관계를 해소하는 교차 테이블이다.

PostgreSQL 접속 확인 (psql 또는 DBeaver)

postgres 에 접속합니다.

psql postgres 명령어를 통해 postgres 에 접속합니다.

CREATE DATABASE

skala_db 라는 이름의 database 를 생성합니다.

```
postgres=# CREATE DATABASE skala_DB
postgres-# WITH ENCODING = 'UTF8'
postgres-# LC_COLLATE = 'en_US.UTF-8'
postgres-# LC_CTYPE = 'en_US.UTF-8'
postgres-# TEMPLATE = template0;
 CREATE DATABASE
```

CREATE SCHEMA

- app, audit 두 개의 schema 를 생성합니다.

```
postgres=# \c skala_db
접속 정보: 데이터베이스="skala_db", 사용자="minjeong".
skala_db=# CREATE SCHEMA app;
CREATE SCHEMA
skala_db=# CREATE SCHEMA audit;
CREATE SCHEMA
skala_db=# SET search_path = app, public;
SET
skala_db=#
```

스마트 데이터의 이해

CREATE TABLE — 제약조건 포함 DDL 작성

Students Table 생성

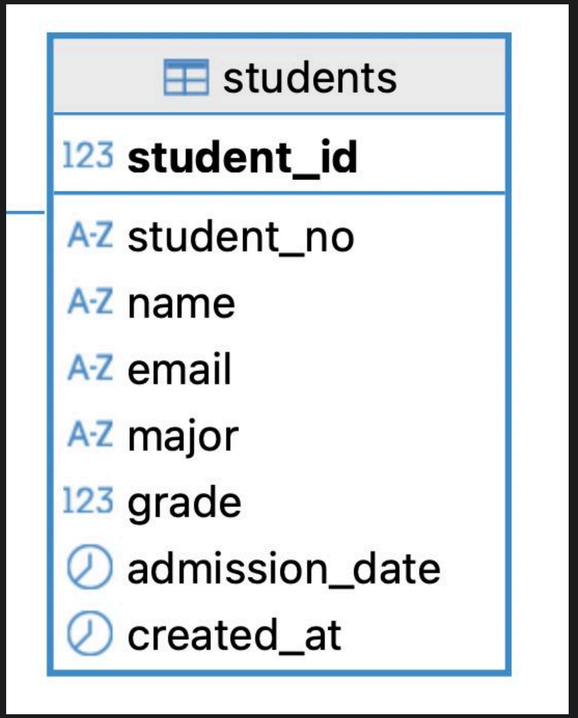
재학 중인 학생의 정보를 관리하는 테이블

요소

- **student_id : PK, 학생을 유일하게 식별하는 기본키.**
- **student_no : 학번**
- **name : 학생 이름**
- **email : 이메일**
- **major : 전공**
- **grade : 학년(1~4학년만 허용)**
- **admission_date : 입학일**
- **created_at : 이 레코드가 DB에 등록된 시각**

```sql
CREATE TABLE students (
    student_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_no      VARCHAR(20) NOT NULL UNIQUE,
    name            VARCHAR(50) NOT NULL,
    email           VARCHAR(100) UNIQUE,
    major           VARCHAR(50) NOT NULL,
    grade           SMALLINT NOT NULL CHECK (grade BETWEEN 1 AND 4),
    admission_date  DATE NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

diagram



CREATE TABLE — 제약조건 포함 DDL 작성

professors 테이블 생성

과목을 담당하는 교수 정보를 관리하는 테이블

요소

- professor_id : PK
- name : 교수 이름, 필수값(NOT NULL)
- department : 소속 학과
- email : 이메일

```
CREATE TABLE professors (
    professor_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name         VARCHAR(50) NOT NULL,
    department   VARCHAR(50) NOT NULL,
    email        VARCHAR(100) UNIQUE
);
```

diagram

CREATE TABLE — 제약조건 포함 DDL 작성

courses 테이블 생성

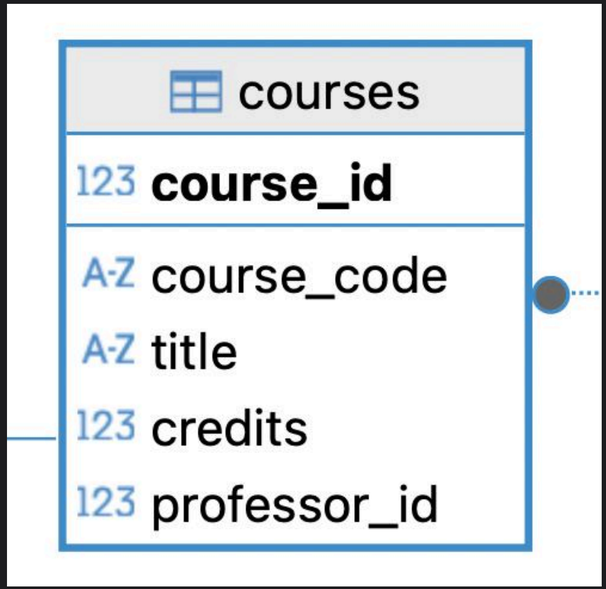
개설된 과목 정보를 관리하는 테이블

요소

- course_id : PK, 과목을 유일하게 식별하는 기본키
- course_code : 과목 코드
- title : 과목명
- credits : 학점 (1~3 사이 값)
- professor_id : FK, professors.professor_id 참조

```sql
CREATE TABLE courses (
    course_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_code  VARCHAR(20) NOT NULL UNIQUE,
    title        VARCHAR(100) NOT NULL,
    credits      SMALLINT NOT NULL CHECK (credits BETWEEN 1 AND 3),
    professor_id BIGINT NOT NULL REFERENCES professors(professor_id) ON DELETE RESTRICT
);
```

diagram



CREATE TABLE — 제약조건 포함 DDL 작성

registration 테이블 생성

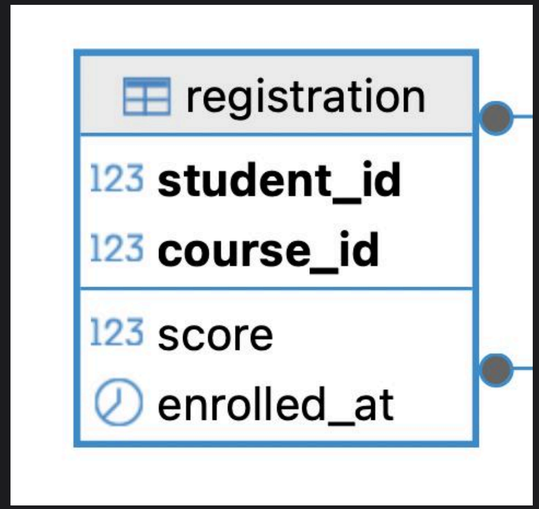
학생이 어떤 과목을 수강 신청했는지 연결하는 테이블

요소

- student_id : FK, students.student_id 참조
- course_id : FK, courses.course_id 참조
- (복합 PK) student_id + course_id : 두 FK를 묶어 기본키로 사용
  같은 학생이 같은 과목을 중복 신청하는 것을 DB에서 차단)
- score : 성적(0~100점). CHECK 제약으로 NULL(미채점) 또는 0~100 사이 값만 허용
- enrolled_at : 수강신청일.
  DEFAULT CURRENT_DATE로 값을 안 넣으면 오늘 날짜가 자동 입력

```sql
CREATE TABLE registration (
    student_id  BIGINT NOT NULL REFERENCES students(student_id) ON DELETE CASCADE,
    course_id   BIGINT NOT NULL REFERENCES courses(course_id)   ON DELETE CASCADE,
    score       NUMERIC(5,2) CHECK (score IS NULL OR score BETWEEN 0 AND 100),
    enrolled_at DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, course_id)
);
```

diagram



INSERT INTO (테이블 데이터는 최소 10건 이상)

- 각 테이블에 최소 10건 이상의 데이터를 입력하고(교수 10건, 강좌 11건, 학생 12건, 수강신청 20건), 전체 건수를 집계하여 검증한다

```sql
INSERT INTO professors (name, department, email) VALUES
('김민준', '컴퓨터공학과', 'kim.prof@skala.ac.kr'),
('이서연', '컴퓨터공학과', 'lee.prof@skala.ac.kr'),
('박지훈', '전자공학과',   'park.prof@skala.ac.kr'),
('최유진', '경영학과',     'choi.prof@skala.ac.kr'),
('정다은', '회계학과',     'jung.prof@skala.ac.kr'),
('오승훈', '컴퓨터공학과', 'oh.prof@skala.ac.kr'),
('한지민', '전자공학과',   'han.prof@skala.ac.kr'),
('서준혁', '경영학과',     'seo.prof@skala.ac.kr'),
('강수지', '회계학과',     'kang.prof@skala.ac.kr'),
('임태양', '컴퓨터공학과', 'lim.prof@skala.ac.kr');

INSERT INTO courses (course_code, title, credits, professor_id) VALUES
('CS101', '프로그래밍기초', 3, 1),
('CS201', '자료구조',       3, 1),
('CS301', '데이터베이스',   3, 2),
('EE101', '회로이론',       3, 3),
('BA101', '경영학원론',     3, 4),
('BA201', '회계원리',       3, 5),
('CS102', '웹프로그래밍',   2, 6),
('EE201', '디지털논리',     3, 7),
('BA301', '마케팅관리',     3, 8),
('AC101', '세무회계',       2, 9),
('CS401', '인공지능개론',   3, 10);

INSERT INTO students (student_no, name, email, major, grade, admission_date) VALUES
('2023001', '홍길동',  'hong@skala.ai',   '컴퓨터공학과', 3, '2023-03-02'),
('2023002', '이순신',  'lee@skala.ai',    '컴퓨터공학과', 3, '2023-03-02'),
('2023003', '강감찬',  NULL,              '컴퓨터공학과', 2, '2024-03-04'),
('2022004', '유관순',  'yu@skala.ai',     '전자공학과',   4, '2022-03-02'),
('2024005', '세종대왕','sejong@skala.ai', '컴퓨터공학과', 1, '2024-03-04'),
('2023006', '장영실',  'jang@skala.ai',   '전자공학과',   3, '2023-03-02'),
('2021007', '신사임당','shin@skala.ai',   '경영학과',     4, '2021-03-02'),
('2023008', '허준',    'heo@skala.ai',    '경영학과',     2, '2023-03-02'),
('2022009', '정약용',  'jung@skala.ai',   '경영학과',     3, '2022-03-02'),
('2024010', '김유신',  'kim@skala.ai',    '전자공학과',   1, '2024-03-04'),
('2023011', '윤봉길',  'yoon@skala.ai',   '컴퓨터공학과', 2, '2023-03-02'),
('2020012', '안중근',  'ahn@skala.ai',    '경영학과',     4, '2020-03-02');

INSERT INTO registration (student_id, course_id, score, enrolled_at) VALUES
(1, 1, 92.5, '2025-03-02'),
(1, 2, 88.0, '2025-09-01'),
(1, 3, NULL, '2026-03-02'),
(2, 1, 76.0, '2025-03-02'),
(2, 3, NULL, '2026-03-02'),
(3, 1, NULL, '2026-03-02'),
(4, 4, 95.0, '2024-09-01'),
(5, 1, NULL, '2026-03-02'),
(6, 4, 73.0, '2025-09-01'),
(8, 5, 81.5, '2025-03-02'),
(9, 6, 67.0, '2025-03-02'),
(10, 4, NULL, '2026-03-02'),
(11, 5, 90.0, '2025-03-02'),
(11, 1, 84.0, '2025-03-02'),
(2, 7, 85.0, '2025-09-01'),
(5, 7, NULL, '2026-03-02'),
(6, 8, 79.0, '2025-09-01'),
(9, 9, 88.5, '2025-03-02'),
(10, 9, NULL, '2026-03-02'),
(3, 11, NULL, '2026-03-02');
```

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	course_id	course_code	title	credits	professor_id
1	1	CS101	프로그래밍기초	3	1
2	2	CS201	자료구조	3	1
3	3	CS301	데이터베이스	3	2
4	4	EE101	회로이론	3	3
5	5	BA101	경영학원론	3	4
6	6	BA201	회계원리	3	5
7	7	CS102	웹프로그래밍	2	6
8	8	EE201	디지털논리	3	7
9	9	BA301	마케팅관리	3	8
10	10	AC101	세무회계	2	9
11	11	CS401	인공지능개론	3	10

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	professor_id	name	department	email
1	1	김민준	컴퓨터공학과	kim.prof@skala.ac.kr
2	2	이서연	컴퓨터공학과	lee.prof@skala.ac.kr
3	3	박지훈	전자공학과	park.prof@skala.ac.kr
4	4	최유진	경영학과	choi.prof@skala.ac.kr
5	5	정다은	회계학과	jung.prof@skala.ac.kr
6	6	오승훈	컴퓨터공학과	oh.prof@skala.ac.kr
7	7	한지민	전자공학과	han.prof@skala.ac.kr
8	8	서준혁	경영학과	seo.prof@skala.ac.kr
9	9	강수지	회계학과	kang.prof@skala.ac.kr
10	10	임태양	컴퓨터공학과	lim.prof@skala.ac.kr

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	student_id	course_id	score	enrolled_at
1	1	1	92.5	2025-03-02
2	1	2	88	2025-09-01
3	1	3	[NULL]	2026-03-02
4	2	1	76	2025-03-02
5	2	3	[NULL]	2026-03-02
6	3	1	[NULL]	2026-03-02
7	4	4	95	2024-09-01
8	5	1	[NULL]	2026-03-02
9	6	4	73	2025-09-01
10	8	5	81.5	2025-03-02
11	9	6	67	2025-03-02
12	10	4	[NULL]	2026-03-02
13	11	5	90	2025-03-02
14	11	1	84	2025-03-02
15	2	7	85	2025-09-01
16	5	7	[NULL]	2026-03-02
17	6	8	79	2025-09-01
18	9	9	88.5	2025-03-02
19	10	9	[NULL]	2026-03-02
20	3	11	[NULL]	2026-03-02

Properties | Data | Diagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	student_id	student_no	name	email	m
1	1	2023001	홍길동	hong@skala.ai	컴퓨터
2	2	2023002	이순신	lee@skala.ai	컴퓨터
3	3	2023003	강감찬	[NULL]	컴퓨터
4	4	2022004	유관순	yu@skala.ai	전자공
5	5	2024005	세종대왕	sejong@skala.ai	컴퓨터
6	6	2023006	장영실	jang@skala.ai	전자공
7	7	2021007	신사임당	shin@skala.ai	경영학
8	8	2023008	허준	heo@skala.ai	경영학
9	9	2022009	정약용	jung@skala.ai	경영학
10	10	2024010	김유신	kim@skala.ai	전자공
11	11	2023011	윤봉길	yoon@skala.ai	컴퓨터
12	12	2020012	안중근	ahn@skala.ai	경영학

스마트 데이터의 이해

SELECT + WHERE + ORDER BY 기초 조회

students 테이블에서 3학년 이상인 학생의 학번, 이름, 전공, 학년을 학번 순으로 조회

```sql
SELECT student_no, name, major, grade
FROM   students
WHERE  grade >= 3
ORDER BY student_no;
```

ents 1

ECT student_no, name, major, grade FR | Enter a SQL expression to filter results (use C

student_no	name	major	grade
2020012	안중근	경영학과	4
2021007	신사임당	경영학과	4
2022004	유관순	전자공학과	4
2022009	정약용	경영학과	3
2023001	홍길동	컴퓨터공학과	3
2023002	이순신	컴퓨터공학과	3
2023006	장영실	전자공학과	3

COALESCE 활용

학생의 이메일이 없으면(NULL) '이메일 미등록'으로 표시해서 전체 학생 조회

```sql
SELECT
    student_no,
    name,
    COALESCE(email, '이메일 미등록') AS email_display
FROM students;
```

students 1

SELECT student_no, name, CC Enter a SQL expression to filter results (use Ctrl+Space)

	student_no	name	email_display
1	2023001	홍길동	hong@skala.ai
2	2023002	이순신	lee@skala.ai
3	2023003	강감찬	이메일 미등록
4	2022004	유관순	yu@skala.ai
5	2024005	세종대왕	sejong@skala.ai
6	2023006	장영실	jang@skala.ai
7	2021007	신사임당	shin@skala.ai
8	2023008	허준	heo@skala.ai
9	2022009	정약용	jung@skala.ai
10	2024010	김유신	kim@skala.ai
11	2023011	윤봉길	yoon@skala.ai
12	2020012	안중근	ahn@skala.ai

스마트 데이터의 이해

CASE WHEN 활용

학생 학년(grade)을 CASE WHEN으로 텍스트 등급명으로 변환해서 조회

```sql
SELECT
    name,
    grade,
    CASE
        WHEN grade = 1 THEN '신입생'
        WHEN grade = 2 THEN '2학년'
        WHEN grade = 3 THEN '3학년'
        WHEN grade = 4 THEN '졸업예정'
    END AS grade_text
FROM students;
```

udents 1

ELECT name, grade, CASE W | Enter a SQL expression to filter results (u

	A-Z name	123 grade	A-Z grade_text
1	홍길동	3	3학년
2	이순신	3	3학년
3	강감찬	2	2학년
4	유관순	4	졸업예정
5	세종대왕	1	신입생
6	장영실	3	3학년
7	신사임당	4	졸업예정
8	허준	2	2학년
9	정약용	3	3학년
10	김유신	1	신입생
11	윤봉길	2	2학년
12	안중근	4	졸업예정

Column: sc
Table: post
Read-only:

스마트 데이터의 이해

날짜 함수 활용

입학일 기준 재학연수를 날짜 함수(AGE, EXTRACT)로 계산해서 조회

```sql
SELECT
    name,
    admission_date,
    EXTRACT(YEAR FROM AGE(CURRENT_DATE, admission_date)) AS 재학연수
FROM students;
```

students 1

SELECT name, admission_dat | Enter a SQL expression to filter results (use Ctrl+Space)

	A-Z name	admission_date	123 재학연수
1	홍길동	2023-03-02	3
2	이순신	2023-03-02	3
3	강감찬	2024-03-04	2
4	유관순	2022-03-02	4
5	세종대왕	2024-03-04	2
6	장영실	2023-03-02	3
7	신사임당	2021-03-02	5
8	허준	2023-03-02	3
9	정약용	2022-03-02	4
10	김유신	2024-03-04	2
11	윤봉길	2023-03-02	3
12	안중근	2020-03-02	6

스마트 데이터의 이해

수강신청 교차 테이블 JOIN 조회

- registration을 중심으로 students, courses, professors 3개 테이블을 JOIN 하여 학생별 수강신청 내역(과목명, 담당교수, 점수, 신청일)을 조회한다.

```sql
SELECT
    s.name        AS student_name,
    c.title       AS course_title,
    p.name        AS professor_name,
    r.score,
    r.enrolled_at
FROM registration r
JOIN students s    ON s.student_id   = r.student_id
JOIN courses c     ON c.course_id    = r.course_id
JOIN professors p  ON p.professor_id = c.professor_id
ORDER BY s.name, c.title;
```

ents(+) 1 ×

ECT s.name AS student_name, c.title AS course_tit | Enter a SQL expression to filter results (use Ctrl+Space)

student_name	course_title	professor_name	score	enrolled_at
허준	경영학원론	최유진	81.5	2025-03-02
강감찬	인공지능개론	임태양	[NULL]	2026-03-02
강감찬	프로그래밍기초	김민준	[NULL]	2026-03-02
김유신	회로이론	박지훈	[NULL]	2026-03-02
김유신	마케팅관리	서준혁	[NULL]	2026-03-02
유관순	회로이론	박지훈	95	2024-09-01
윤봉길	경영학원론	최유진	90	2025-03-02
윤봉길	프로그래밍기초	김민준	84	2025-03-02
이순신	데이터베이스	이서연	[NULL]	2026-03-02
이순신	웹프로그래밍	오승훈	85	2025-09-01
이순신	프로그래밍기초	김민준	76	2025-03-02
장영실	회로이론	박지훈	73	2025-09-01
장영실	디지털논리	한지민	79	2025-09-01
정약용	회계원리	정다은	67	2025-03-02
정약용	마케팅관리	서준혁	88.5	2025-03-02
홍길동	자료구조	김민준	88	2025-09-01
홍길동	데이터베이스	이서연	[NULL]	2026-03-02
홍길동	프로그래밍기초	김민준	92.5	2025-03-02
세종대왕	웹프로그래밍	오승훈	[NULL]	2026-03-02
세종대왕	프로그래밍기초	김민준	[NULL]	2026-03-02